

Zero to Hero Study Handbook: agentic-document-extraction

Module 1: Foundations & Architecture

1.1 What This Project Does

`agentic-document-extraction` is a full-stack system that:

- uploads documents (`/api/documents/`)
- defines extraction schemas (`/api/schemas/`)
- runs an asynchronous LangGraph extraction pipeline (`/api/extractions/`)
- validates output and optionally routes to human review
- exposes provider readiness/model catalogs (`/api/providers/*`)
- offers an MCP server interface (`backend/app/mcp_server.py`) for tool-based LLM clients.

Main use cases from code and docs:

- extracting structured fields from PDFs and images
- handling multiple OCR/parser engines (`auto` , `paddleocr` , `glmocr` , `docling` + internal `pymupdf`)
- choosing LLM providers (`auto` , `openai` , `gemini` , `anthropic`)
- supporting review workflows (`approved` , `corrected` , `rejected`)
- recording observability, metrics, and audit events.

1.2 Core CS Definitions and Patterns Used Here

1. `Asynchronous programming` - Definition: Concurrency model where I/O waits do not block the whole process. - Here: most backend paths are `async def` (`FastAPI` , `SQLAlchemy AsyncSession` , `LangGraph async stream` , `async job queue methods`).
2. `Event-driven architecture` - Definition: System state advances because events occur (HTTP requests, pipeline node updates, SSE emissions, review submissions). - Here: extraction node updates are streamed and persisted step-by-step in `backend/app/routers/extractions.py` via `extraction_graph.astream(..., stream_mode="updates")` .
3. `Finite-state workflow / state machine` - Definition: Computation is represented by states and transitions. - Here: extraction lifecycle statuses in `ExtractionStatus` (`pending` , `queued` , `processing` , `ocr_complete` , `extracted` , `completed` , `needs_review` , `failed`) and LangGraph node transitions in `backend/app/services/extraction/graph.py` .
4. `Typed contract-driven design` - Definition: explicit schemas for input/output and invariants. - Here: Pydantic models in `backend/app/models/schemas.py` plus `field_validator` , `model_validator` , `computed_field` .

5. `Strategy pattern` - Definition: interchangeable implementations behind a shared interface. - Here: - OCR strategies implement `BaseOCRProvider` (`pymupdf`, `paddleocr`, `glmocr`, `docling`) - LLM strategies implement `BaseLLMProvider` (`openai`, `gemini`, `anthropic`) - registries choose concrete strategy by policy (`backend/app/services/ocr/registry.py`, `backend/app/services/llm/registry.py`).
6. `Human-in-the-loop pattern` - Definition: automation defers uncertain/invalid outputs to human decision. - Here: `review_verdict == "needs_review"` plus review endpoints `POST /api/extractions/{id}/reviews`.
7. `Checkpoint/resume workflow` - Definition: save intermediate state and resume later. - Here: LangGraph supports `interrupt()` in `await_review_node`; optional SQLite checkpoint path via `Settings.checkpoint_db_path`.

1.3 Architecture and Interactions

Main runtime topology

```
[Next.js Frontend]
|
| HTTP /api/* (rewritten by frontend/next.config.js)
v
[FastAPI app.main]
|-- routers/documents.py --> save_upload + MIME validation --> documents table
|-- routers/schemas.py --> schema CRUD/presets -->
extraction_schemas table
|-- routers/providers.py --> parser/provider/model metadata
|-- routers/extractions.py
|
| create_extraction() -> queue.submit()
v
[Job Queue]
|-- InProcessJobQueue (default)
|-- ArqJobQueue (if REDIS_URL set)
v
_run_extraction_job()
v
LangGraph extraction_graph
  triage -> parse -> extract -> validate -> reflect -> await_review ->
finalize
|
v
Persist to DB:
  extractions, extraction_steps, extraction_reviews,
  extraction_judgments, extraction_audit_log, v0.5 tables
|
+--> SSE stream /api/extractions/{id}/stream
+--> REST status/result/validation endpoints
```

1.4 Technology Stack with Real Config Values

Backend core:

- FastAPI app version "0.6.0" (backend/app/main.py)
- SQLAlchemy[asyncio] +aiosqlite default DB URL:
- database_url = "sqlite+aiosqlite:///./extraction.db" (backend/app/config.py)
- LangGraph pipeline in backend/app/services/extraction/graph.py
- SlowAPI limiter default: RATE_LIMIT_DEFAULT = "60/minute" (backend/app/constants.py)

- Prometheus metrics endpoint `/metrics` with metric names like `ade_extractions_total`, `ade_llm_call_duration_seconds`.

Job execution:

- `job_max_concurrent = 8` default (`Settings.job_max_concurrent`)
- queue factory in `get_job_queue()`:
- `InProcessJobQueue` when `REDIS_URL` empty
- `ArqJobQueue` when `REDIS_URL` is set
- Redis queue key for arq path: `"ade:extractions:queue"` (`backend/app/services/jobs.py`)

Pipeline tuning:

- `confidence_threshold = 0.6`
- `llm_max_retries = 2`
- `llm_retry_base_delay = 1.0`
- `max_reflection_attempts = 2`
- `job_timeout_s = 300` via `JOB_TIMEOUT_S`
- SSE constants:
- `SSE_KEEPALIVE_S = 15.0`
- `SSE_MAX_ITERATIONS = 600`
- terminal statuses: `{"completed", "needs_review", "failed"}`.

Provider defaults:

- LLM auto priority order: `openai -> gemini -> anthropic` (`AUTO_PRIORITY` in LLM registry)
- LLM default model IDs:
- OpenAI: `gpt-4o-mini`
- Gemini: `gemini-2.0-flash`
- Anthropic: `claude-3-5-haiku-20241022`
- LLM extraction temperature: `0` in all three provider classes.

OCR routing:

- OCR auto priority in `backend/app/services/ocr/registry.py`:
- `glmocr -> paddleocr -> docling -> pymupdf`
- Feature flags:
- `enable_paddleocr = False`
- `enable_glm_ocr = False`
- `enable_docling = False`
- internal parser:
- `pymupdf` is `is_user_selectable = False`, PDF-only fallback.

Frontend:

- Next.js 14 (frontend/package.json)
- API rewrite:
- /api/:path* -> http://localhost:8000/api/:path* (frontend/next.config.js)

Security and network guard:

- security headers from SECURITY_HEADERS
- local-only OLLAMA guard at startup via validate_ollama_base_url()
- opt-out env var for remote host: OLLAMA_ALLOW_PRIVATE_HOSTS=true .

Module 2: Repository Map

Focus: files new contributors should learn first.

File/Directory Path	Primary Responsibility	Key Classes/ Functions	Important Configs/ Variables
pyproject.toml	Python package/deps/ tooling metadata	[project] , [tool.pytest] , [tool.ruff]	name="agentic- document-extraction", version="0.6.0" , requires- python=">=3.12,<3.13"
justfile	task runner recipes for install/dev/test/migrate	install , sync , dev , migrate , mcp	uv venv --python 3.12.10 , uv sync -- frozen --extra test --extra lint --extra ollama
backend/.env.example	environment template	key list only	OPENAI_API_KEY , GEMINI_API_KEY , ANTHROPIC_API_KEY , DEFAULT_LLM_PROVIDER , DATABASE_URL , ENABLE_* , OLLAMA_*
backend/app/main.py	FastAPI app boot, middleware, health/ info routes	lifespan , readiness , health_check , app_info	app version="0.6.0" , startup recovery, router registration
backend/app/config.py	Pydantic settings object	class Settings	job_max_concurrent=8 , llm_max_retries=2 , confidence_threshold= 0.6 , redis_url=""
backend/app/ constants.py	cross-cutting constants	module constants	RATE_LIMIT_DEFAULT="6 0/minute" , JOB_TIMEOUT_S=300 , SSE_KEEPALIVE_S=15.0

File/Directory Path	Primary Responsibility	Key Classes/ Functions	Important Configs/ Variables
backend/app/database.py	async DB engine/ session/init lifecycle	get_db , init_db , close_db	startup Alembic attempt + fallback create_all , PRAGMA journal_mode=WAL
backend/app/models/enum.py	wire-format enums for API/DB	ParserEngine , LLMProviderID , ExtractionStatus , etc.	stable values used by backend + frontend
backend/app/models/schemas.py	API request/response contracts	ExtractionCreate , ExtractionResponse , ReviewCreate , AppInfoResponse	validators, computed fields duration_total_ms , validation_summary
backend/app/models/db_models.py	ORM entities/tables	Document , Extraction , ExtractionStep , ExtractionReview , etc.	table names, column types/defaults, relationships
backend/app/routers/documents.py	upload/list/get/delete docs	upload_document , list_documents	MIME sniff + extension match verification
backend/app/routers/schemas.py	extraction schema CRUD + presets	create_schema , create_schema_from_preset , update_schema	uniqueness checks, preset materialization
backend/app/routers/extractions.py	extraction job API, retry, SSE, review	create_extraction , _run_extraction_pipeline , stream_extraction_progress , submit_review	_PIPELINE_STEPS , _JOB_TIMEOUT , status/ error transitions
backend/app/routers/providers.py	parser/LLM/model/ config metadata endpoints	get_parser_options , get_llm_providers , get_llm_models , get_app_config	cache TTLS CACHE_MAX_AGE_* , parser/provider serialization
backend/app/services/extraction/graph.py	LangGraph pipeline nodes and routing	trriage_node , parse_node , extract_node , validate_node , reflect_node , await_review_node , finalize_node	_MAX_LLM_RETRIES , _RETRY_BASE_DELAY , max_reflection_attempts

File/Directory Path	Primary Responsibility	Key Classes/ Functions	Important Configs/ Variables
backend/app/services/extraction/validation.py	field-level extraction validation	validate_extraction , compute_review_verdict	confidence-aware review routing
backend/app/services/extraction/business_rules.py	built-in deterministic rules loaded at startup	module import side-effects	startup import in lifespan
backend/app/services/jobs.py	queue abstraction and implementations	InProcessJobQueue , ArqJobQueue , get_job_queue	settings.redis_url , Redis list key ade:extractions:queue
backend/app/services/ocr/registry.py	OCR provider registration + auto policy	get_ocr_provider , list_ocr_provider_statuses	AUTO_PRIORITY=("glmocr", "paddleocr", "docling", "pymupdf")
backend/app/services/ocr/pymupdf_provider.py	internal PDF parser	PyMuPDFProvider.extract_text	is_user_selectable=False , supported_file_types= {"pdf" }
backend/app/services/ocr/paddleocr_provider.py	local image OCR adapter	PaddleOCRProvider.extract_text	supports PaddleOCR 2.x and 3.x paths
backend/app/services/ocr/glm_ocr_provider.py	Ollama GLM-OCR adapter	GLMOCRProvider.extract_text	OLLAMA_BASE_URL , OLLAMA_GLM_OCR_MODEL , timeout settings
backend/app/services/ocr/docling_provider.py	Docling structured parser adapter	DoclingProvider.extract_text	feature_flag_name="enable_docling"
backend/app/services/llm/registry.py	LLM provider registration + auto policy	get_llm_provider , list_models_for_provider	auto priority + default provider fallback logic
backend/app/services/llm/openai_provider.py	OpenAI extraction/ model listing	OpenAIProvider.extract_text	default model gpt-4o-mini , temperature=0
backend/app/services/llm/gemini_provider.py	Gemini extraction/ model listing	GeminiProvider.extract_text	default model gemini-2.0-flash , temperature=0
backend/app/services/llm/claude_provider.py	Anthropic extraction/ model listing	ClaudeProvider.extract_text	default model claude-3-5-haiku-20241022 , temperature=0

File/Directory Path	Primary Responsibility	Key Classes/ Functions	Important Configs/ Variables
backend/app/services/llm/prompts_loader.py	versioned prompt loading	load_prompt , list_prompts , index	prompt dirs prompts/v1 , prompts/v2 , env override ADE_PROMPTS_DIR
prompts/v1/*.md , prompts/v2/*.md	prompt templates + metadata	front-matter + body	extraction.md , reflection.md
backend/app/metrics.py	Prometheus metrics registry	class Metrics , render	metric names prefixed ade_*
backend/app/security_middleware.py	response security headers	SecurityHeadersMiddleware	headers from SECURITY_HEADERS
backend/app/mcp_server.py	MCP stdio server and tool handlers	tools: extract_document , verify_extraction , resolve_entities , eval_golden_set	SERVER_NAME="agentic-document-extraction"
backend/alembic/env.py	Alembic migration runner	run_migrations_online /offline	uses settings.database_url , render_as_batch=True
backend/alembic/versions/0001_initial_schema.py	initial schema migration	Alembic revision	creates core tables
backend/alembic/versions/0002_judgments.py	judgments migration	Alembic revision	adds extraction_judgments
backend/alembic/versions/0003_prompt_schema_version.py	metadata columns migration	Alembic revision	adds prompt_version , schema_version
backend/alembic/versions/0004_evidence_entities_verifier.py	v0.5 evidence tables	Alembic revision	adds evidence/entity/verifier tables
frontend/src/lib/api.ts	frontend API client + TS contracts	request<T>() , API methods, enums/ interfaces	mirrors backend enums, error normalization
frontend/src/app/page.tsx	main UX page flow	HomePage	3-stage flow: upload -> configure -> review

File/Directory Path	Primary Responsibility	Key Classes/ Functions	Important Configs/ Variables
frontend/src/components/ExtractionResult.tsx	live job status/results UI	ExtractionResult	SSE stream + polling fallback, retry support
frontend/src/components/ReviewPanel.tsx	human review submission UI	component + handlers	decisions approved/corrected/rejected
frontend/next.config.js	API routing between FE and BE	rewrites()	/api/* proxy destination
docker-compose.yml	local multi-service stack	services app , phoenix , ollama	healthcheck path / health/ready , OTEL/ Ollama env wiring
docs/ARCHITECTURE_V2.md	architecture deep dive	narrative docs	pipeline reasoning and design
docs/DEVELOPMENT.md	contributor setup workflow	commands/instructions	backend/.env workflow, uv usage
docs/MCP.md	MCP setup and client integration	run/config examples	stdio launch patterns

Module 3: Core Execution Flows

3.1 Boot Sequence (Backend)

Start point: backend/app/main.py (lifespan).

Step-by-step:

1. configure_logging()
2. validate_ollama_base_url(settings.ollama_base_url) rejects non-local hosts unless OLLAMA_ALLOW_PRIVATE_HOSTS=true .
3. await init_db() : - tries alembic upgrade head - falls back to Base.metadata.create_all - applies SQLite PRAGMA journal_mode=WAL and PRAGMA optimize .
4. ensures directories by touching settings.upload_path , settings.artifacts_path .
5. best-effort telemetry setup (app.telemetry.setup_telemetry) .
6. imports app.services.extraction.business_rules at startup.
7. logs provider readiness via list_llm_provider_statuses() and list_ocr_provider_statuses() .
8. calls _recover_orphaned_jobs() : - statuses queued/processing/ocr_complete/extracted are failed on restart - stuck running ExtractionStep rows are finalized as failed.

3.2 Document Upload Flow

Endpoint: `POST /api/documents/ (upload_document)`.

Execution path:

1. `save_upload(file)` writes file to `UPLOAD_DIR`.
2. verifies magic bytes with `sniff_mime(Path(file_path))`.
3. checks extension/type agreement via `mime_matches_extension(...)`.
4. creates `Document` ORM row: - `id`, `filename`, `original_filename`, `file_path`, `file_type`, `file_size`, `status`.
5. returns `DocumentResponse`.

`DocumentResponse` shape:

```
{
  "id": "string",
  "filename": "string",
  "original_filename": "string",
  "file_type": "string",
  "file_size": 12345,
  "page_count": null,
  "status": "uploaded",
  "created_at": "ISO datetime"
}
```

3.3 Extraction Start + Background Job Flow

Endpoint: `POST /api/extractions/ (create_extraction)`.

Request model `ExtractionCreate`:

```
{
  "document_id": "32-char id",
  "schema_id": "32-char id",
  "ocr_provider": "auto|paddleocr|glmocr|docling",
  "llm_provider": "auto|openai|gemini|anthropic",
  "llm_model": "auto or model id"
}
```

Flow:

1. validate `Document` and `ExtractionSchema` existence.
2. insert `Extraction` with `status="queued"`.
3. increment metrics (`ade_in_flight_jobs`, `ade_extractions_total{status="queued"}`).
4. enqueue worker callback using `queue.submit(extraction.id, lambda: _run_extraction_job(...))`.

5. audit event `extraction.started`.

3.4 LangGraph Pipeline Flow (Core Logic)

Orchestrator: `_run_extraction_pipeline(extraction_id) + extraction_graph.astream`.

Node order (from code):

```
triage -> parse -> extract -> validate -> reflect -> await_review -> finalize
```

State object: `PipelineState` (`TypedDict`) with key fields:

- inputs: `file_path`, `schema_fields`, `ocr_provider_id`, `llm_provider_id`, `llm_model_id`
- parse output: `ocr_text`, `ocr_provider_used`
- extract output: `extracted_data`, `llm_provider_used`, `llm_model_used`, `confidence`, `extract_attempts`
- validate output: `validation_errors`, `validation_results`, `review_verdict`
- finalize output: `status`, `completed_at`, `error`.

Important node behavior:

1. `trriage_node` - chooses recommendation based on file suffix (pdf/image/office/html), respects explicit provider selection.
2. `parse_node` - gets OCR provider via `get_ocr_provider(...)` - returns `{"status":"ocr_complete", "ocr_text":..., "ocr_provider_used":...}` or `{"status":"failed","error":...}`.
3. `extract_node` - resolves provider via `get_llm_provider(...)` - retries retryable provider errors up to `settings.llm_max_retries` - exponential backoff base `settings.llm_retry_base_delay` - returns `status="extracted"` with extracted dict + confidence.
4. `validate_node` - runs `validate_extraction(...)` - computes verdict via `compute_review_verdict(...)` - populates both structured `validation_results` and plain `validation_errors`.
5. `reflect_node` - if verdict not valid and attempts remain, rebuilds prompt with `build_reflection_prompt(...)` and re-extracts.
6. `await_review_node` - when checkpointer exists, uses LangGraph `interrupt(payload)` and resumes with: - `{"decision":"approved|corrected|rejected","corrected_fields": {...},"notes":"..."}` - merges corrections into `extracted_data` for `corrected`.
7. `finalize_node` - sets terminal `status`: - `completed` when `review_verdict == "valid"` - `needs_review` when still unresolved.

3.5 Persisted Step Records and SSE

`_PIPELINE_STEPS` in router:

```
("trriage", "parse", "extract", "validate", "reflect", "await_review", "finalize")
```

Each streamed node update:

- completes current `ExtractionStep` with `status`, `duration_ms`, `error`
- creates next step row as `running`
- updates `Extraction.status` progressively.

Live stream endpoint: `GET /api/extractions/{extraction_id}/stream`

- implementation uses `StreamingResponse` with `text/event-stream`
- emits only on status/step changes using `_sse_step_signature(...)`
- frame format from `_sse_event` :

```
data: {"id":"...", "status":"processing", ...}
```

3.6 Review Flow

Endpoint: `POST /api/extractions/{id}/reviews` with `ReviewCreate`.

Rules in code:

- only when extraction status is `needs_review`
- `decision="corrected"` requires non-empty `corrected_fields`
- `corrected_fields` is forbidden for `approved / rejected`.

Outcome mapping:

- `approved` -> extraction `status="completed"`
- `corrected` -> merge `corrected_fields` into result then complete
- `rejected` -> extraction `status="failed"`.

3.7 Provider Metadata Flow

Endpoints:

- `GET /api/providers/parsers`
- `GET /api/providers/llm`
- `GET /api/providers/llm/{provider_id}/models`
- `GET /api/providers/config`

`/providers/config` real payload fields:

- `parser_engines`
- `llm_providers`
- `default_llm_provider`
- `model_selection_modes`
- `ocr_engine_flags` (`paddleocr`, `glm_ocr`)

- `max_upload_size_mb`
- `supported_file_types`
- `confidence_threshold`.

Module 4: Setup & Run Guide

This section is a static setup guide derived from repo files (`README.md` , `docs/DEVELOPMENT.md` , `justfile` , `backend/.env.example`).

4.1 Clean-Machine Install Steps

1. Create and activate environment (Python 3.12.10 pinned in `docs/justfile`):

```
uv venv --python 3.12.10 .venv
source .venv/bin/activate
```

1. Install dependencies (backend + extras):

```
uv pip install -e ".[test,lint,ollama]"
```

1. Frontend install:

```
cd frontend
npm install
```

4.2 Environment Configuration

Template source: `backend/.env.example` .

Minimum required keys for extraction:

- one LLM API key:
- `OPENAI_API_KEY` or `GEMINI_API_KEY` or `ANTHROPIC_API_KEY`

Important core keys:

- `DEFAULT_LLM_PROVIDER=auto`
- `DATABASE_URL=sqlite+aiosqlite:///./extraction.db`
- `UPLOAD_DIR=./uploads`
- `ARTIFACTS_DIR=./artifacts`
- `MAX_UPLOAD_SIZE_MB=50`
- `ENABLE_PADDLEOCR=false`
- `ENABLE_GLM_OCR=false`
- `OLLAMA_BASE_URL=http://localhost:11434`
- `OLLAMA_GLM_OCR_MODEL=glm-ocr:latest`
- `GLM_OCR_TIMEOUT_SECONDS=120`

- `HOST=0.0.0.0`
- `PORT=8000`
- `DEBUG=true`
- `CORS_ORIGINS=http://localhost:3000,http://127.0.0.1:3000`

Additional settings present in `backend/app/config.py` (not all in example file):

- `ENABLE_DOCLING`
- `ENABLE_LAYOUT_PARSING`
- `ENABLE_VERIFIER`
- `ENABLE_DOUBLE_PASS`
- `ENABLE_CROSS_PAGE_ENTITIES`
- `REDIS_URL`
- `CONFIDENCE_CALIBRATION_PATH`
- `CHECKPOINT_DB_PATH`
- `OTEL_*`
- `VLM_*`
- `JUDGE_*`.

4.3 Typical Command Sequences

Backend dev server:

```
uvicorn app.main:app --reload --port 8000 --app-dir backend
```

Frontend dev server:

```
cd frontend
npm run dev
```

Justfile equivalents:

- backend dev: `just dev`
- DB migration: `just migrate`
- stack up: `just up`
- MCP server: `just mcp`.

4.4 Database Migration and Seeding Notes

Migration system:

- Alembic config: `alembic.ini`
- migration env: `backend/alembic/env.py`
- versions:
- `0001_initial_schema.py`

- 0002_judgments.py
- 0003_prompt_schema_version.py
- 0004_evidence_entities_verifier.py.

Migration command:

```
alembic upgrade head
```

Seeding:

- no dedicated DB seed script is defined in repo root.
- schema presets are code-defined and materialized through API (/api/schemas/presets then POST /api/schemas/presets/{preset_id}).

Module 5: Study Plan & Practice Exercises

5.1 Ordered Self-Study Plan

Recommended read order:

1. README.md and docs/ARCHITECTURE_V2.md for context.
2. backend/app/models/enums.py , backend/app/models/schemas.py , backend/app/models/db_models.py to internalize contracts.
3. backend/app/main.py , backend/app/config.py , backend/app/constants.py for runtime boot/config.
4. Routers in this order: - documents.py - schemas.py - providers.py - extractions.py
5. Pipeline internals: - services/extraction/graph.py - services/extraction/validation.py - services/extraction/business_rules.py
6. Providers: - services/ocr/registry.py + OCR providers - services/llm/registry.py + LLM providers
7. Infra and ops: - services/jobs.py - database.py - metrics.py - security_middleware.py
8. Frontend integration: - frontend/src/lib/api.ts - frontend/src/app/page.tsx - frontend/src/components/ExtractionResult.tsx - frontend/src/components/ReviewPanel.tsx
9. Migrations and MCP: - backend/alembic/* - backend/app/mcp_server.py .

5.2 Practice Exercises (with answer outlines)

1. Explain full flow of POST /api/extractions/ from request body to DB persistence and queue submission.
2. List every terminal extraction status and identify which functions assign them.
3. Compare OCR auto-routing vs explicit OCR selection behavior and error conditions.
4. Trace how review_verdict moves from validation node to final API response fields.
5. Document the exact SSE stop condition and bounded-loop behavior.

6. Identify where and how model retries are implemented for LLM calls.
7. Map all places that can mark a job as failed outside normal graph node failure.
8. Find a backend/frontend contract mismatch and explain impact.
9. Describe how startup protects against unsafe remote Ollama endpoints.
10. Show which tables were added after the initial migration and why they matter.

5.3 Model Answer Outlines

1. `create_extraction` validates `document_id + schema_id`, inserts extraction row (`status="queued"`), increments metrics, submits queue callback, writes audit event `extraction.started`, returns `ExtractionResponse`.
2. Terminal statuses are `completed`, `needs_review`, `failed`. Assigned in: - `finalize_node` (`completed` or `needs_review`) - `_apply_failure_state` and `_mark_job_failed` (`failed`) - review submission path maps rejected review to `failed`.
3. Auto OCR calls `_resolve_auto(file_path)` and walks priority `glmocr -> paddleocr -> docling -> pymupdf` with checks: enabled, available, file-type compatibility. Explicit selection bypasses auto and errors if disabled/unavailable/incompatible.
4. `validate_node` computes `review_verdict`; `reflect_node` may improve extraction and loop back; `await_review_node` can convert to valid via approve/correct; persisted by router into `extraction.review_verdict`; returned in `ExtractionResponse.review_verdict` and `ExtractionValidationResponse.review_verdict`.
5. SSE endpoint loops up to `SSE_MAX_ITERATIONS` (`600`), sleeps `SSE_KEEPALIVE_S` (`15.0`), emits only when status or step signature changes, exits early on terminal statuses set in `SSE_TERMINAL_STATUSES`.
6. `extract_node` catches `LLMProviderError`, retries only when `exc.retryable` and `attempt < max`; delay is `llm_retry_base_delay * (2**attempt)`; cap is `settings.llm_max_retries`.
7. Failures also occur in: - `_run_extraction_job` timeout wrapper (`JOB_TIMEOUT_S=300`) - `_mark_job_failed` on crash - startup `_recover_orphaned_jobs` for stuck statuses after restart - parse/extract node exception paths.
8. One real mismatch: backend enum includes `docling` (`ParserEngine.DOCLING`), but frontend `ParserEngine` union in `frontend/src/lib/api.ts` omits `DOCLING`; this can break strong typing for parser option handling.
9. `main.lifespan` calls `validate_ollama_base_url`; non-loopback host is rejected unless `OLLAMA_ALLOW_PRIVATE_HOSTS` is true.
10. Post-initial migration tables include: - `extraction_judgments` (`0002`) - `prompt_version` and `schema_version` columns (`0003`) - `extraction_evidence`, `extraction_entities`, `extraction_verifier_runs` (`0004`) for evidence-grounded/verifier pipeline capabilities.

| Learner Verification Checklist

Use this checklist after reading:

- Can you explain how an extraction transitions from `queued` to terminal status, including queue + graph + DB persistence?
- Can you name each LangGraph node and describe exactly what state keys it writes?
- Can you explain OCR auto-selection policy and why `pymupdf` is internal-only?
- Can you describe how `ReviewCreate` validation rules enforce corrected vs non-corrected decisions?
- Can you trace where retries/backoff happen and what config keys control them?
- Can you list the major DB tables and which API endpoints read/write them?
- Can you explain how SSE progress events are generated and terminated?
- Can you describe startup safety checks (DB init, provider readiness, OLLAMA URL policy, orphaned-job recovery)?
- Can you point to at least one contract drift between backend and frontend and explain remediation?
- Can you set up a clean machine using `uv`, env keys, migrations, backend serve, and frontend dev commands without guessing?